Práctica 3: Java RMI

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
24 de abril de 2026

Índice

1. Introducción **2**

2. Parte 1. Ejemplos del guion. **2**

2.1. Ejemplo 1. Cliente-Servidor básico. 2

2.1.1. `Ejemplo_I.java` (La Interfaz Remota) 3

2.1.2. `Ejemplo.java` (El Servidor) 3

2.1.3. `Cliente_Ejemplo.java` (El Cliente) 5

2.1.4. `launch.sh` (Script de Ejecución) 5

2.1.5. Ejemplo de compilación y ejecución del ejemplo 1. . . . 8

1. Introducción

En esta tercera práctica vamos a trabajar y aprender el manejo de los mecanismos para implementar programas distribuidos en Java, haciendo uso de su API **RMI** (***R**emote **M**ethod **I**nvocation*), que nos permite construir objetos distribuidos. Como entorno de funcionamiento se utilizará la máquina virtual de java (*JVM*, Java Virtual Machine) que sigue un enfoque *"Write Once, Run Anywhere"*, que su traducción viene a ser: "se codifica una vez y se ejecuta en cualquier sitio"), usando este enfoque para que se ejecute en ó por todas partes (*"run everywhere"*).

Los sistemas RMI se caracterizan porque existe un objeto delegado (que llamaremos *stub)*) que es el encargado de gestionar las invocaciones al objeto remoto, gestionando la serialización y deserialización de los parámetros y valores de retorno.

Las aplicaciones en RMI (denominadas **aplicaciones de objetos distribuidos**)están formadas por dos aplicaciones: el cliente y el servidor. El servidor es el encargado de crear los objetos remotos, hacer accesibles las referencias a dichos objetos remotos y esperar la invocación de métodos sobre dichos objetos remotos por parte de los clientes. Estos, los clientes, obtienen una referencia remota a uno o varios objetos remotos en el servidor e invocan métodos sobre los objetos remotos. RMI proporciona los mecanismos que permiten la comunicació y el intercambio de información entre el cliente y el servidor.

A continuación, veremos los tres ejemplos que nos dan en el guion de este práctica, los ejecutaremos y analizaremos que está sucediendo, así como mostrar sus archivos, pues considero que es más cómodo a la hora de corregir, tenerlo todo en un documento (hablo de tener el código fuente de los ejercicios más ejemplos de ejecución), que tener la necesidad de abrir el documento y el editor de código a la vez.

2. Parte 1. Ejemplos del guion.

2.1. Ejemplo 1. Cliente-Servidor básico.

Este ejemplo es una muestra de cómo se realiza una implementación básica de una aplicación Cliente-Servidor en Java RMI. Tenemos un servidor llamado `Ejemplo` que implementa un método `escribir_mensaje(int idProceso)`, que recibe un entero que representa el identificador del proceso (Cliente) que invoca a este método del servidor, y muestra por pantalla dicho identificador. Si se trata del proceso/hebra 0, entonces la duerme 5 segundos

y después continua como el resto de procesos.

Veamos cada uno de los archivos que forman este ejemplo. Es importante señalar que **este código ha sido modificado durante las sesiones de prácticas**, pues se trata de una práctica antigua y hay paquetes/componentes que actualmente están en estado obsoletos.

2.1.1. `Ejemplo_I.java` (La Interfaz Remota)

Es la interfaz que define qué métodos del servidor pueden ser llamados por el cliente a través de la red.

Su código es:

```
1  // Ejemplo_I.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
7  public interface Ejemplo_I extends Remote {
8      public void escribir_mensaje (int id_proceso) throws RemoteException;
9  }
```

2.1.2. `Ejemplo.java` (El Servidor)

Implementa la interfaz anteriormente definida, es decir, el método `escribir_mensaje`. Esta aplicación está a la espera de recibir peticiones. Aquí se han **realizado modificaciones**, ya que el `SecurityManager` del ejemplo está obsoleto (*deprecated*) a partir de Java 17, por lo que se han añadido las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` para que la comunicación vaya cifrada a través de **SSL/TSL**.

Su código es:

```
1  // Ejemplo.java
2  // Implementa la Interfaz remota
3
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.registry.Registry;
6  import java.rmi.server.UnicastRemoteObject;
7  import javax.rmi.ssl.SslRMIClientSocketFactory;
8  import javax.rmi.ssl.SslRMIServerSocketFactory;
9
10 public class Ejemplo implements Ejemplo_I {
11     private static final int RMI_PORT = 2099;
12
```

```
13      public Ejemplo() {
14          super();
15      }
16
17      public void escribir_mensaje (int id_proceso) {
18          System.out.println("Recibida peticion de proceso: "+id_proceso);
19          if (id_proceso == 0) {
20              try{
21                  System.out.println("Empezamos a dormir");
22                  Thread.sleep(5000);
23                  System.out.println("Terminamos de dormir");
24              }
25              catch (Exception e) {
26                  System.err.println("Ejemplo exception:");
27                  e.printStackTrace();
28              }
29          }
30          System.out.println("\nHebra "+id_proceso);
31      }
32
33      public static void main(String[] args) {
34          try {
35              // Definimos el nombre del objeto remoto que se registrará en el ligador
36              String nombre_objeto_remoto = "Ejemplo_I";
37
38              // Creamos una instancia del objeto remoto
39              Ejemplo_I prueba = new Ejemplo();
40
41              // Creamos los socket factories para SSL
42              SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
43              SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
44
45              // Creamos el stub del objeto remoto para el cliente, exportandolo al
46              // puerto 0 (puerto anonimo) y usando los socket factories creados.
47              Ejemplo_I stub =
48                  (Ejemplo_I) UnicastRemoteObject.exportObject(prueba, 0, csf, ssf);
49
50              // El Registry vive como proceso externo (rmiregistry).
51              Registry registry = LocateRegistry.createRegistry(RMI_PORT);
52
53              // Registramos el objeto remoto en el ligador
54              registry.rebind(nombre_objeto_remoto, stub);
55
56              // A partir de aquí, el objeto remoto ya puede ser invocado por los clientes.
57
58              System.out.println("Ejemplo bound");
59          }
60          catch (Exception e) {
61              System.err.println("Ejemplo exception:");
```

```
            e.printStackTrace();
        }
    }
}
```

2.1.3. `Cliente_Ejemplo.java` (El Cliente)

Es el encargado de conectar con el servidor para realizar peticiones. Para ello, se conecta al registro (el `rmiregistry`) que hace de guía para poder encontrar nuestros objetos, y busca el servidor por su nombre, que sería ".Ejemplo_I". Después se llama al método para escribir el mensaje, pasándole su identificador de proceso como parámetro.

Su código sería el siguiente:

```
// Cliente_Ejemplo.java
// Código del cliente que invoca al objeto remoto implementado en Ejemplo.java

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Cliente_Ejemplo {
    private static final int RMI_PORT = 2099;

    public static void main(String args[]) {
        try {
            String nombre_objeto_remoto = "Ejemplo_I";
            System.out.println("Buscando el objeto remoto");
            Registry registry = LocateRegistry.getRegistry(args[0], RMI_PORT);
            // Invocamos el objeto remoto obtenido del ligador
            Ejemplo_I instancia_local = (Ejemplo_I) registry.lookup(nombre_objeto_remoto);
            System.out.println("Invocando el objeto remoto");
            // Llamamos al método remoto, pasándole
            // el id del proceso (recibido como argumento)
            instancia_local.escribir_mensaje(Integer.parseInt(args[1]));
        } catch (Exception e) {
            System.err.println("Ejemplo_I exception:");
            e.printStackTrace();
        }
    }
}
```

2.1.4. `launch.sh` (Script de Ejecución)

Nos han proporcionado este script para poder realizar la compilación/ejecución con tan sólo ejecutarlo. Básicamente, lo que hace es crear

unos “certificados digitales” necesarios para que el SSL funcione en neustra máquina, compilar todos los .java de golpe, lanzar el servidor en segundo plano y por último lanzar dos clientes para que se conecten al servidor, estos clientes tienen como identificadores 0 y 3, respectivamente.

```
#!/bin/sh -e
# ejecutar = Macro para compilación y ejecución del programa ejemplo
# en una sola maquina Unix de nombre localhost.

KEYSTORE=server-keystore.p12
TRUSTSTORE=client-truststore.p12
STOREPASS=changeit
RMI_PORT=2099

if [ ! -f "$KEYSTORE" ]; then
        echo
        echo "Generando keystore TLS de ejemplo ..."
        keytool -genkeypair \
                -alias rmi-server \
                -keyalg RSA \
                -keysize 2048 \
                -validity 3650 \
                -dname "CN=localhost, OU=RMI, O=DSD, L=Local, ST=Local, C=ES" \
                -storetype PKCS12 \
                -keystore "$KEYSTORE" \
                -storepass "$STOREPASS" \
                -keypass "$STOREPASS" \
                -noprompt
fi

if [ ! -f "$TRUSTSTORE" ]; then
        echo
        echo "Generando truststore TLS de ejemplo ..."
        keytool -exportcert \
                -alias rmi-server \
                -keystore "$KEYSTORE" \
                -storetype PKCS12 \
                -storepass "$STOREPASS" \
                -rfc \
                -file rmi-server.crt

        keytool -importcert \
                -alias rmi-server \
                -file rmi-server.crt \
                -storetype PKCS12 \
                -keystore "$TRUSTSTORE" \
                -storepass "$STOREPASS" \
                -noprompt
```

```
44
45          rm -f rmi-server.crt
46  fi
47
48  # Si quedo un proceso previo en el puerto, lo cerramos para evitar conflictos.
49  if command -v fuser >/dev/null 2>&1; then
50          fuser -k "${RMI_PORT}/tcp" >/dev/null 2>&1 || true
51  fi
52
53  echo
54  echo "Compilando con javac ..."
55  javac *.java
56
57  sleep 2
58
59  echo
60  echo "Lanzando el servidor"
61  java -cp . \
62          -Djava.rmi.server.hostname=localhost \
63          -Djavax.net.ssl.keyStore="$KEYSTORE" \
64          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
65          -Djavax.net.ssl.keyStoreType=PKCS12 \
66          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
67          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
68          -Djavax.net.ssl.trustStoreType=PKCS12 \
69          Ejemplo &
70
71  sleep 2
72
73  echo
74  echo "Lanzando el primer cliente"
75  echo
76  java -cp . \
77          -Djavax.net.ssl.keyStore="$KEYSTORE" \
78          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
79          -Djavax.net.ssl.keyStoreType=PKCS12 \
80          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
81          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
82          -Djavax.net.ssl.trustStoreType=PKCS12 \
83          Cliente_Ejemplo localhost 0
84
85  sleep 2
86
87  echo
88  echo "Lanzando el segundo cliente"
89  echo
90  java -cp . \
91          -Djavax.net.ssl.keyStore="$KEYSTORE" \
92          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
```

```
93          -Djavax.net.ssl.keyStoreType=PKCS12 \
94          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
95          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
96          -Djavax.net.ssl.trustStoreType=PKCS12 \
97          Cliente_Ejemplo localhost 3
```

2.1.5. Ejemplo de compilación y ejecución del ejemplo 1.

En primer lugar, debemos dar permiso de ejecución al script. Para ello abrimos una terminal y ejecutamos el siguiente comando:

```
$ chmod u+x launch.sh
```

Después, ejecutamos el script con el siguiente comando:

```
$ ./launch.sh
```

A continuación, muestro una imagen con la salida de la ejecución:

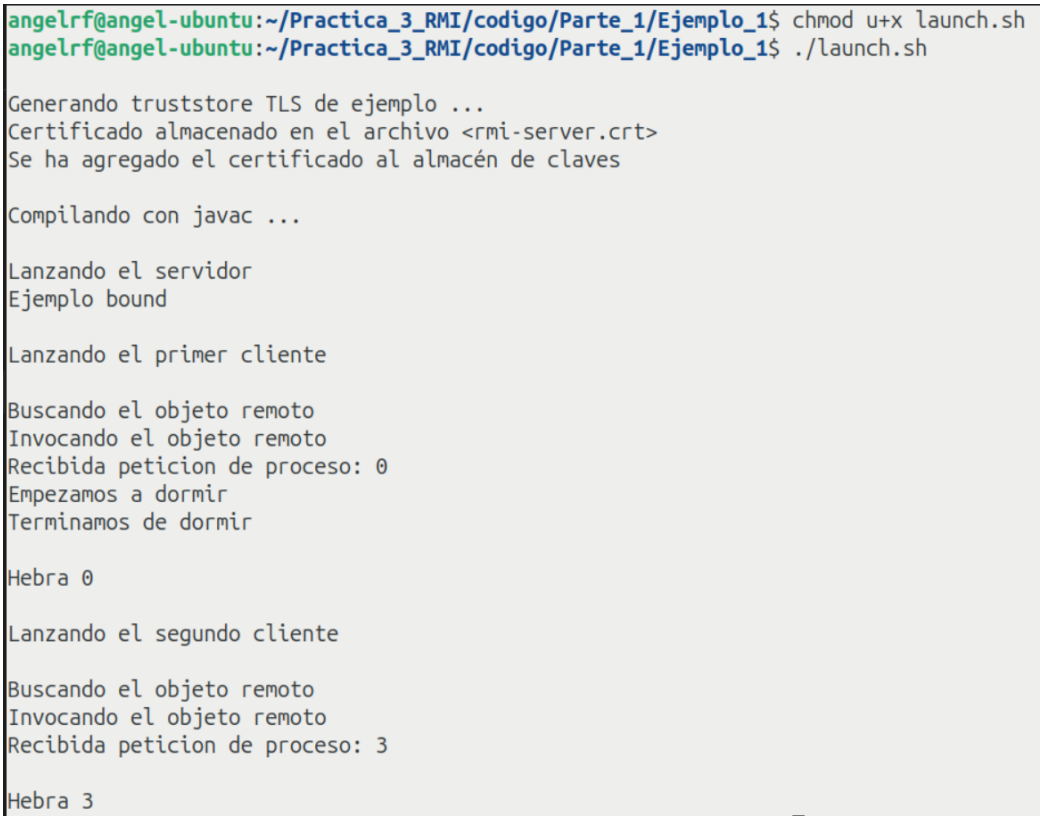


Figura 1: Ejecución del ejemplo 1

Como podemos observar en la salida, se compilan todos los archivos, se lanza el servidor y se muestra el mensaje `.Ejemplo bound"`, que nos indica que el objeto remoto ya puede ser invocado por los clientes. Después se lanza el cliente 0. Este busca el objeto remoto y lo invoca. El método de escribir mensaje en el servidor recibe la petición mostrando ese mensaje con el id del proceso cliente. Como se trata del proceso 0, lo pone a dormir 5 segundos. Pasado ese tiempo, muestra el mensaje de que ha terminado de dormir e imprime rl mensaje de hebra con el id de nuevo, y ya termina. Finalmente se lanza el cliente con id 3 (aunque no se lanza después de que termine la hebra 0, si no que **se lanza la hebra 3 mientras la hebra 0 duerme**).De nuevo, este cliente busca el objeto remoto y lo invoca. Llama al método para escribir mensaje, este muestra que le ha llegado la petición mostrando su id.

2.2. Ejemplo 2. Multihebra.